INTERVIEW PRACTICE • TWO PROBLEMS

01 Edit the Filename

Core concept: strings + dynamic programming (minimum edit distance).

Problem statement

A student has built an application that receives a **filename** and a **dictionary string**. Find the minimum number of single-character operations required to transform the filename into the dictionary string. If both are already equal, output **0**.

An operation has unit cost and may be performed at any position:

- **Insert:** add one character to the filename.
- **Remove:** delete one character from the filename.
- **Replace:** change one character in the filename to another character.

Input, output and constraints

Input: line 1 contains the filename; line 2 contains the dictionary string. **Output:** print one integer - the minimum number of operations.

Both strings contain only letters and digits. Each length is greater than 1 and at most 25.

One test case

SAMPLE INPUT

```
Hello
There
```

EXPECTED OUTPUT

```
5
```

Idea to solve (2D DP)

- Let **dp[i][j]** be the minimum edits to turn the first **i** characters of **file** into the first **j** characters of **dict**.
- Initialize the borders: **dp[i][0] = i** (remove all characters) and **dp[0][j] = j** (insert all characters).
- If the current characters are equal, use **dp[i][j] = dp[i-1][j-1]**. Otherwise use **1 + min(dp[i-1][j], dp[i][j-1], dp[i-1][j-1])**.
- The answer is **dp[n][m]**. Time: **O(n × m)**; space: **O(n × m)**.

> **Important sample inconsistency:** the provided statement says the strings are case-insensitive, but its expected output for **Hello → There** is **5**, which is obtained with **case-sensitive** comparison. Converting both strings to lowercase gives 4. Follow the judge sample when practicing this specific task.

Similar problems to practice

LeetCode 72 - Edit Distance - near-exact algorithmic match; three edit operations.

GeeksforGeeks - Edit Distance - recursion, memoization, tabulation and optimized DP.

INTERVIEW PRACTICE • TWO PROBLEMS

02 Mario and the Number Mountain

Core concept: matrix neighborhoods + sorting + pattern construction.

Problem statement

Mario stands at the top-left of an **n × n** grid of integers. A special **loophole** is located at the middle cell of the grid. To reach Princess Peach, Mario must use the values around that cell to build a mountain of numbers. Print the mountain according to the rules below.

Rules for building the mountain

- Set the loophole value to the center cell, **arr[n/2][n/2]** (zero-based indexing).
- If **n ≤ 9**, consider the **8 surrounding cells** (3 × 3 region excluding the center). If **n > 9**, consider the **24 surrounding cells** (5 × 5 region excluding the center).
- Add the center value to each selected neighboring cell. Include **0** for the center itself. This gives 9 or 25 values.
- Sort these values from largest to smallest. The **middle element** of the sorted list becomes the mountain tip.
- For each subsequent level, pick the next values immediately to the **left and right** of the middle sorted position as the row edges. Fill the gaps alternately with the **difference (left - right)** and **sum (left + right)** of those edges, starting with the difference.

Input, output and constraints

Input: the first line gives the odd integer **n**, followed by **n** rows containing **n** space-separated integers. **Output:** print the mountain, one level per line, with space-separated values.

Constraints: n is odd and n > 2. The number of levels is 5 (3 × 3 region) or 13 (5 × 5 region).

One test case

SAMPLE INPUT

```
5
1 2 1 9 6
3 3 9 8 5
4 1 2 3 2
1 1 2 6 7
7 8 9 0 3
```

EXPECTED OUTPUT

```
5
5 4
8 5 3
10 7 13 3
11 11 11 11 0
```

> **What the sample tests:** the center value is **2**; the eight neighboring cells are **3, 9, 8, 1, 3, 1, 2, 6**. After adding 2 and including center 0, they become **5, 11, 10, 3, 5, 3, 4, 8, 0**.

PROBLEM 02 • SOLUTION IDEA

Build the Mountain Step by Step

Use the sample on the previous page to trace the construction.

Approach

- **1. Find the center:** use **mid = n / 2**, then **center = arr[mid][mid]**.
- **2. Collect nearby values:** select offsets from **-1 to +1** when n ≤ 9, otherwise from **-2 to +2**. Push **arr[i][j] + center** into a vector for every non-center cell, and push **0** for the center.
- **3. Sort descending:** for the sample, **v = [11, 10, 8, 5, 5, 4, 3, 3, 0]**.
- **4. Locate the middle: m = v.size() / 2**. Output **v[m]** for the first line. Here, **v[4] = 5**.
- **5. Generate each remaining level:** for level **i = 1, 2, ... m**, set **left = v[m-i]** and **right = v[m+i]**. Print **left**, then **i-1** interior entries, then **right**. Interior positions 1, 3, ... use **left - right**; 2, 4, ... use **left + right**.

Quick walkthrough

Level	Chosen edges	Printed row
0	tip = 5	5
1	5 and 4	5 4
2	8 and 3	8 5 3
3	10 and 3	10 7 13 3
4	11 and 0	11 11 11 11 0

Complexity: reading the complete matrix takes $O(n^2)$. The selected neighborhood contains only k = 9 or 25 values, so sorting takes $O(k \log k)$. Printing the triangle takes $O(k^2)$ operations in this bounded formulation. Use a wide integer type if input values can be large.

Related practice problems online

LeetCode 661 - Image Smoother - practice reading the eight neighbors around a matrix cell.

GFG - Sort Array in Spiral Manner From Center - practice arranging sorted values around a middle position.

GFG - Pattern Printing Problems - practice using nested loops to print triangular number patterns.

> **Match quality:** these three links cover individual techniques used in the Mario question, **not** the same complete mountain-building problem. An exact public match was not verified.

Suggested revision order: first solve Edit Distance without looking at the recurrence. Then implement 3 × 3 neighborhood collection, sort the results, and finally write the mountain-printing loop.